MediQueue: An AI-Assisted Emergency Department Queue Built on Priority Scheduling and Process Synchronization

Aahan Anand
Dept. of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

Ayush Anupam Sahu
Dept. of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

Ankit Dhupal
Dept. of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

Source code: https://github.com/anandaahan7-prog/mediqueue

***Abstract*—Most small and mid-sized hospitals in India still run the emergency waiting area with a paper register or a token machine, so patients are mostly seen in order of arrival. This paper presents MediQueue, a web-based queue and triage system that treats the emergency department as a small operating system: doctors are the processors and waiting patients form the ready queue. A multilevel priority dispatcher with aging decides whom the next free doctor sees, and preemption is allowed only for resuscitation cases. Because several nurses and doctors use the system at the same moment, dispatching a patient and allocating an observation bed are implemented as critical sections, with the beds modelled as a counting semaphore. Three machine learning models support the staff: a random forest that suggests a triage level from vital signs, a second random forest that predicts consultation time, and a gradient boosting model that corrects the waiting time shown to patients. On 3,000 synthetic cases the triage suggestion reached 82.8 percent accuracy with 5.2 percent under-triage, against 55.3 percent accuracy for a NEWS2-only rule. Over 40 simulated twelve-hour shifts the scheduler reduced the mean wait of ESI 1–2 patients to 9.4 minutes, compared with 27.0 minutes under first-come-first-served, while holding the 95th percentile wait of low-priority patients at 206 minutes instead of 355 minutes under plain priority scheduling. A concurrency stress test produced 45 double assignments without locking and none with it.**

***Keywords*—operating systems, CPU scheduling, aging, process synchronization, semaphore, emergency triage, random forest, waiting-time prediction**

I. Introduction

Anyone who has waited in a hospital casualty ward knows the routine: you take a token and wait for your number to be called. It is simple and it feels fair, but it is not how emergency medicine is supposed to work. A patient with chest pain and falling blood pressure should not sit behind three people with sprained ankles only because they came in later. Larger hospitals deal with this through formal triage, in which a trained nurse grades every patient on a scale such as the five-level Emergency Severity Index (ESI) [4]. Even there, the grade usually lives on a paper slip, and the order in which doctors call patients still depends on whoever is standing closest to the desk.

While studying CPU scheduling in our operating systems course, we noticed that the casualty ward is a scheduling problem in a very literal sense. There are a few processors (the doctors), a ready queue that keeps growing (the waiting room), jobs with different priorities and unknown burst times, and the familiar risk of starvation, because a low-priority patient can be overtaken again and again. There is also genuine concurrency, since the nurses at the triage desk and the doctors in three or four rooms all work on the same queue at the same time.

Our first prototype tried to show as many OS topics as possible, including the Banker's algorithm and a deadlock detector. When we reviewed it as a team we had to admit that most of those features did not correspond to any decision a real emergency department makes; they were there because they were in the syllabus. For this version we kept only two concepts, the ones a hospital would actually need if it started using the software tomorrow: priority scheduling with aging to dispatch patients to doctors, and synchronization for the operations that several staff members perform at once. The AI features were rebuilt around the same idea, so that each model answers a question the staff really ask. The main contributions of this work are:

- A dispatcher that runs a five-level priority queue with aging, capped at level 3, on several doctors at once, with preemption limited to ESI 1 patients.
- Critical sections for patient dispatch and bed allocation built on the database write lock, with observation beds treated as a counting semaphore and boarding patients as its blocked queue.
- Three AI models: a triage suggestion with a safety margin and red-flag rules, a consultation-time predictor, and a learned correction for the waiting time shown in the waiting room.
- A working web application with screens for nurses, doctors, the waiting room and the administrator, plus an offline simulator that uses the same scheduling code for evaluation.

Section II reviews related work and Section III lists the requirements. Section IV presents the system design, Section V the two OS components and Section VI the AI modules. Section VII describes the implementation, Section VIII the results, and Section IX concludes the paper.

II. Literature Survey

A. CPU Scheduling and Synchronization

Standard textbooks [1], [2] describe priority scheduling together with its main weakness, indefinite blocking or starvation, and the usual remedy, aging, in which the priority of a waiting process is raised as time passes. When several processors share one ready queue, the queue itself needs mutual exclusion. Dijkstra's semaphore [3] is still the

standard tool for this, and a counting semaphore is the natural model when a resource has several identical units. Kleinrock's analysis of priority queues [5] showed that strict priority for one class makes the waits of lower classes grow sharply as the load approaches capacity, which is exactly the starvation problem of an overcrowded ward.

B. Triage Scales and Machine Learning

The ESI algorithm [4] sorts patients into five levels using danger signs and the number of resources a patient is expected to need. NEWS2 [6] is a simpler score built from six physiological measurements and the level of consciousness, and is widely used to detect deterioration on hospital wards. Several groups have trained machine learning models for triage. Levin et al. [7] reported that a random-forest based electronic triage tool separated patients by outcome better than ESI, and Hong et al. [8] predicted hospital admission at triage with gradient boosting and deep learning. Both used large hospital datasets. MIMIC-IV-ED [9] is public, but it requires credentialed access, so we built a synthetic dataset instead (Section VI-A).

C. Waiting Time and Patient Flow

Ang et al. [10] showed that simple queue-length estimates of emergency waiting time are often badly wrong, and that combining queueing quantities with a regression model reduces the error considerably. Saghafian et al. [11] reviewed operations research on patient flow and found that prioritisation rules strongly affect both waiting times and safety, and their complexity-augmented triage [12] argued that expected treatment time, not only urgency, should be considered when ordering patients. These papers directly shaped our use of predicted consultation time and our learned correction of the waiting-time estimate.

D. Existing Systems

Table I compares MediQueue with what hospitals commonly use. Token display systems manage the queue but know nothing about urgency, and the triage modules of hospital information systems (HIS) record a triage level but rarely use it to decide the calling order or to tell patients how long they will wait.

TABLE I.
COMPARISON WITH EXISTING APPROACHES

Feature	**Token / register**	**HIS triage**	**MediQueue**
Urgency-based calling order	No	Partial	Yes
Protection against starvation	–	No	Aging
Many staff on one queue	Manual	Yes	Critical section
Triage decision support	No	Rules	AI + red flags
Reason for the suggestion	No	No	Yes
Estimated wait for patients	No	No	ML-corrected
Queue for observation beds	No	Partial	Semaphore

III. Requirement Analysis

A. Functional Requirements

- Login with three roles (nurse, doctor, administrator); the waiting-room display needs no login.
- Registering a patient with age, sex, six vital signs, consciousness (AVPU), pain score and complaint, and issuing a token.
- An AI triage suggestion with reasons, which the nurse can accept or override with a written reason.
- Re-triage of a waiting patient and marking patients who leave without being seen.
- "Call next patient" for doctors, and pause-and-attend for resuscitation cases.
- Discharge or admission to an observation bed; release and cleaning of beds.
- Reports, scheduler settings and a concurrency test for the administrator.

B. Non-Functional Requirements

Correctness under concurrency matters most: a patient must never be given to two doctors and a bed must never be given to two patients. The AI suggestion has to appear while the nurse is still typing, so it should respond well under 100 ms. Patient names must not appear on the public display. Finally, the system has to run on an ordinary laptop without a separate database server, so that a small hospital could try it without IT support.

C. Hardware and Software Requirements

Any PC with 4 GB of RAM is sufficient. The server uses Python 3.12, Flask 3.1 [17], SQLite 3, scikit-learn 1.8 [15], NumPy and joblib; the client uses HTML, CSS and plain JavaScript. Matplotlib and Playwright were used only for the evaluation and for this report.

D. Use Case Model

Fig. 1 shows the four actors. The nurse registers and triages patients, reviews the AI suggestion, re-triages waiting patients and manages beds. The doctor calls the next patient, pauses for a resuscitation case, and discharges or admits patients. The administrator views reports, changes the aging interval and runs the concurrency test. Patients and visitors only see the waiting-room display.

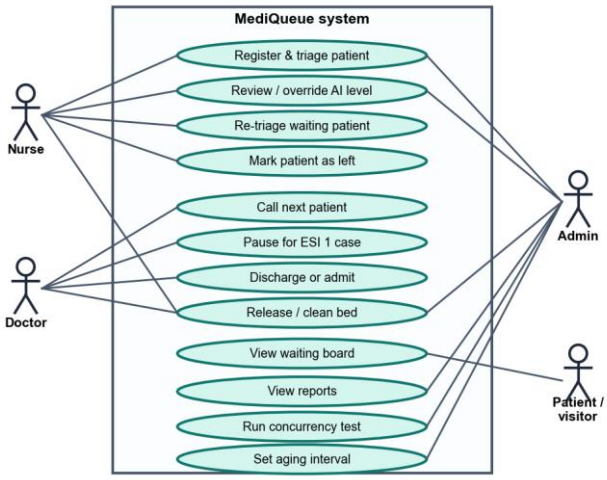


Fig. 1. Use case diagram.

IV. System Design

A. System Architecture

MediQueue uses a three-tier architecture (Fig. 2). Each screen in the presentation tier polls a JSON endpoint every five seconds, so a token issued at the triage desk appears on the doctors' consoles and the waiting-room display almost

immediately. The Flask application tier runs every request on its own thread, and this is precisely why the two OS components are needed: the dispatcher and the bed manager are called from many threads at the same time. The AI models are trained offline and loaded from joblib files when the server starts. SQLite forms the data tier; we switched it to write-ahead logging (WAL) so that screens that only read data do not block the writers.

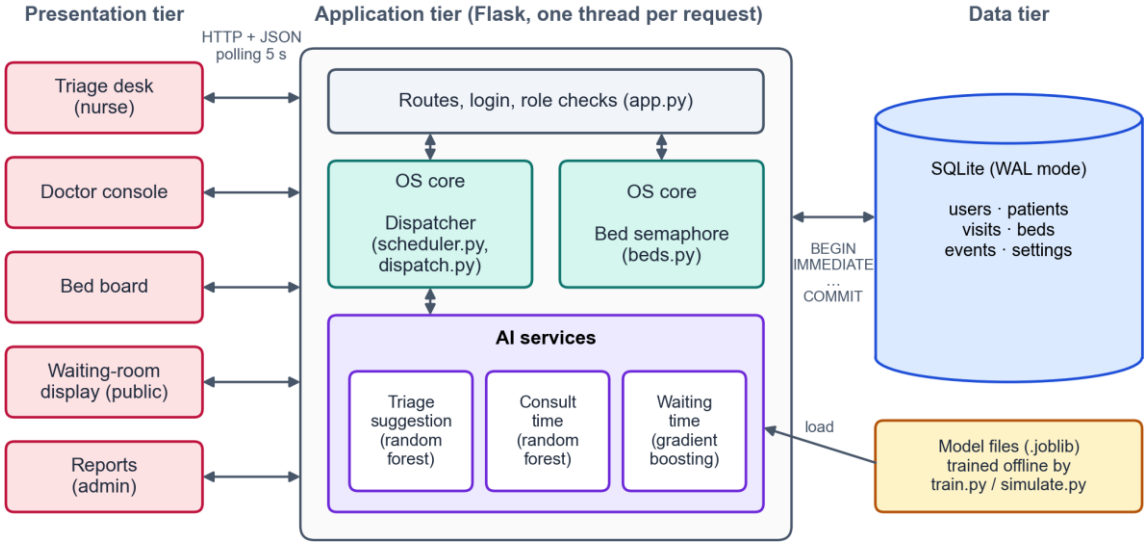


Fig. 2. Three-tier architecture of MediQueue.

B. Data Flow Diagrams

The context diagram (Fig. 3) shows the whole system as one process exchanging data with the nurse, the doctor, the administrator and the patient. The level-1 DFD (Fig. 4) splits it into six processes: triage patient, AI suggestion, dispatch patient, manage beds, estimate waits, and reports and settings. They use five data stores: visits and patients, beds, events, model files, and users and settings. The model file store appears twice only to keep the diagram readable.

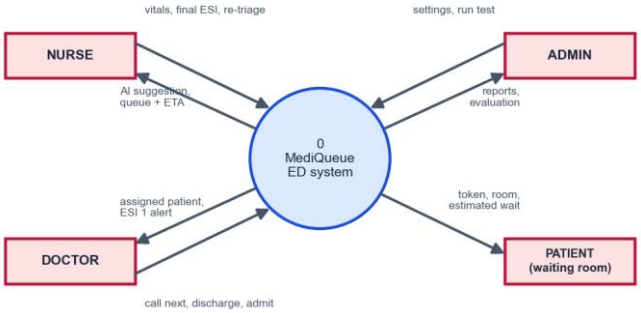


Fig. 3. Level-0 (context) data flow diagram.

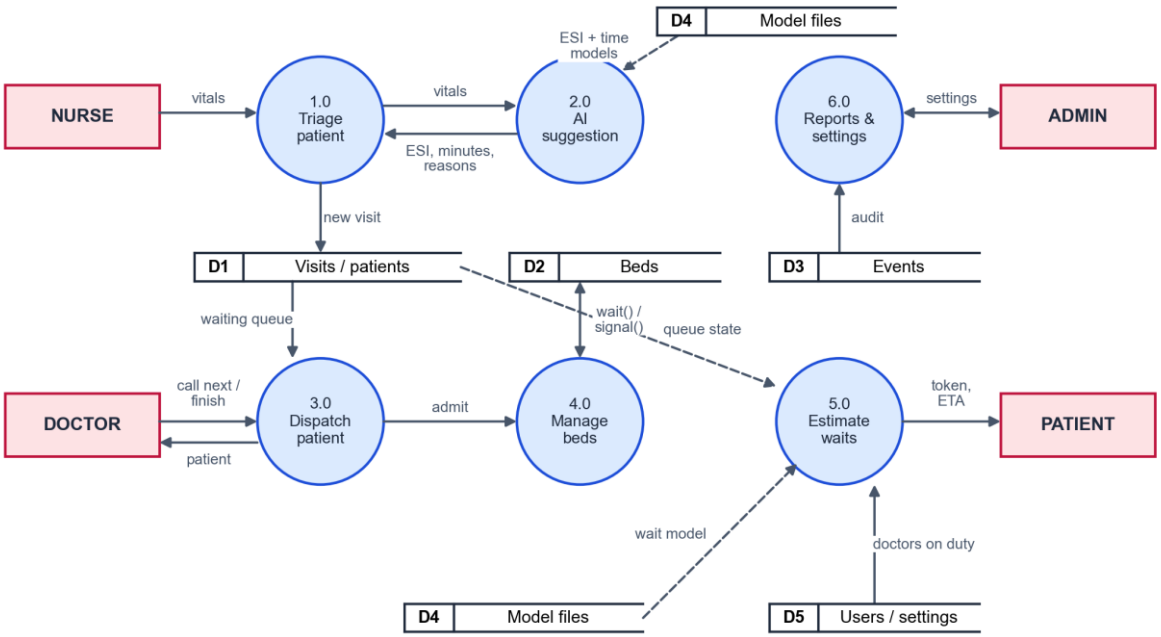


Fig. 4. Level-1 data flow diagram.

C. Database Design

The schema (Fig. 5) has six tables. The visits table is the central one and plays the role of a process control block: it holds everything the scheduler needs about a patient, namely arrival time, final triage level, predicted consultation time, status, the assigned doctor and the minutes of consultation already completed. Storing both ai_esi and esi lets us measure how often nurses disagree with the model. A UNIQUE constraint on beds.visit_id guarantees at database level that one patient can never hold two beds, and CHECK constraints restrict status, role, ESI level, SpO2 and pain score to valid values. Table II lists the tables of the demonstration database.

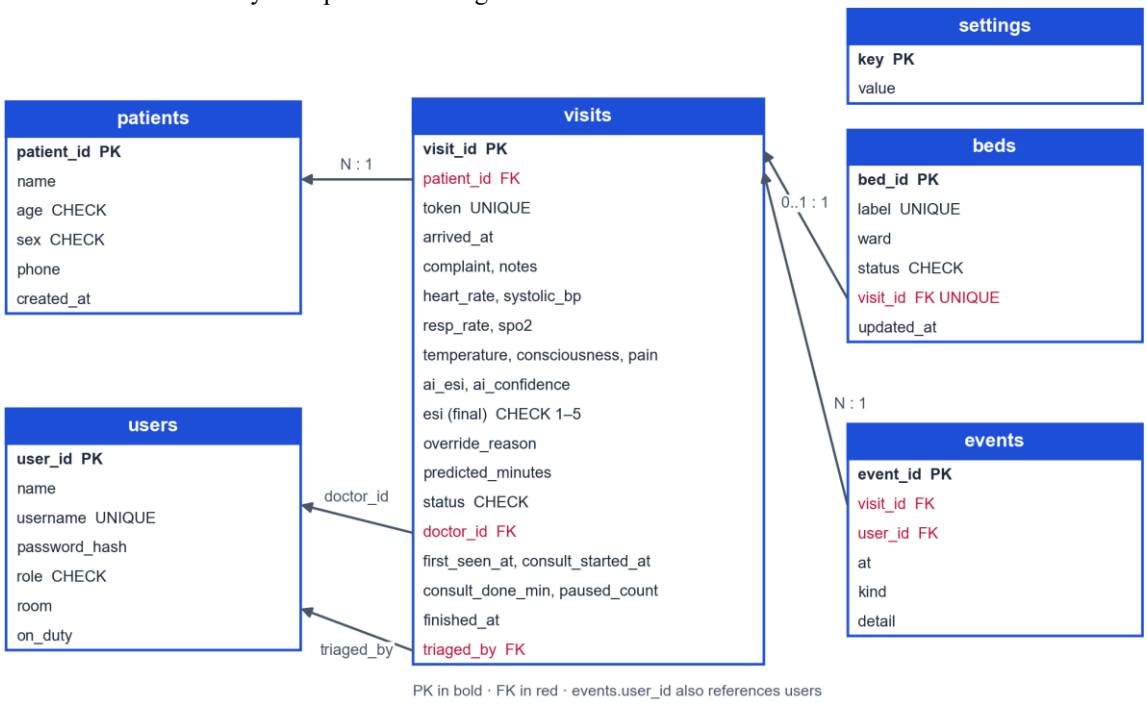


Fig. 5. Relational schema with primary keys, foreign keys and constraints.

TABLE II.
TABLES IN THE DATABASE

Table	Primary key	Foreign keys	Rows
users	user_id	–	7
patients	patient_id	–	38
visits	visit_id	patient_id, doctor_id, triaged_by	38
beds	bed_id	visit_id (unique)	8
events	event_id	visit_id, user_id	38
settings	key	–	1

D. Visit Life Cycle

Fig. 6 shows the states that a visit passes through. We named them after process states on purpose, because the mapping is almost exact: a waiting patient is ready, a patient with a doctor is running, and an admitted patient who is still waiting for a bed is blocked on the bed semaphore. Table III summarises how each OS idea appears in the system.

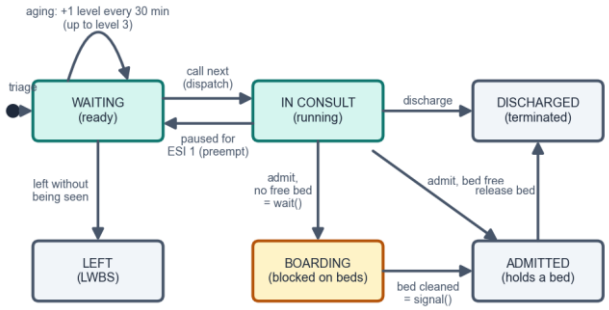


Fig. 6. Life cycle of a visit, drawn as process states.

TABLE III.
OS CONCEPTS AND THEIR ROLE IN MEDIQUEUE

OS concept	MediQueue equivalent
Process / PCB	Visit / row in visits
Processor	Doctor in a consultation room
Ready queue	Waiting room (status = waiting)
Running / blocked	In consultation / boarding
Priority	Final ESI level (1 = highest)
Burst time	Predicted consultation minutes
Aging	+1 level per 30 min, up to level 3
Preemption	Pause, for ESI 1 patients only
Critical section	Call next, admit, mark bed clean
Counting semaphore	Free observation beds

V. OS Concepts in MediQueue

A. Dispatching Patients to Doctors

Dispatch is pull-based: the scheduler runs only when a doctor presses “Call next patient”. This matches how

consultation rooms already work, and it means the scheduler needs no background thread. For every waiting patient p with final level $e(p)$ and waiting time $w(p)$ in minutes, the effective level is

$$level(p) = max(3,\ e(p) - \lfloor w(p) / A \rfloor) \qquad (1)$$

for $e(p) \geq 3$, while patients with $e(p) \leq 2$ keep their level. A is the aging interval, 30 minutes by default. The doctor receives the patient with the smallest pair (level(p), arrival time), so patients on the same level are served first-come-first-served.

Two details came from mistakes in our earlier prototype. At first we allowed every patient to age all the way to level 2. Under heavy load almost everybody reached level 2 within an hour and the queue quietly turned into FCFS, so the critical patients lost the advantage the whole design was supposed to give them. Capping aging at level 3 keeps levels 1 and 2 for patients whom triage actually flagged as high risk. The aging interval itself was first a guess; Section VIII-C shows the sweep we used to settle on 30 minutes.

Consultations are otherwise non-preemptive. Pulling a doctor out of a consultation has a real cost, so we allow it only for ESI 1. If a resuscitation patient is waiting and every doctor on duty is busy, the dispatcher picks the doctor whose current patient is the least urgent (ESI 3 or lower), and that doctor sees a red banner asking them to pause. The paused patient keeps the original arrival time and the minutes already spent, so they return near the front of their level and their remaining time is still predicted correctly. If all doctors are treating ESI 1 or 2 patients, nobody is interrupted and the banner asks for backup instead. Fig. 7 shows the complete flow.

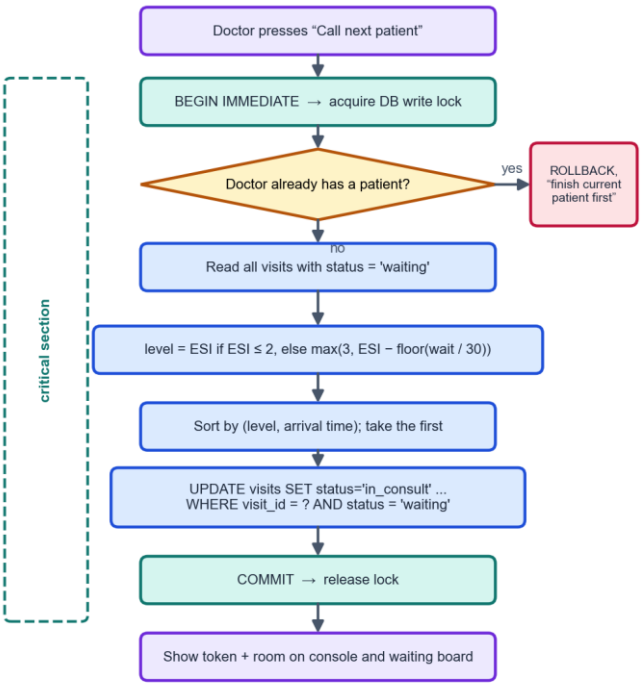


Fig. 7. Flow of "Call next patient". Everything between BEGIN IMMEDIATE and COMMIT is one critical section.

B. Dispatch as a Critical Section

"Call next" is a read-modify-write sequence: read the waiting queue, choose a patient, mark that patient as taken. When two doctors press the button a few milliseconds apart, the sequence runs on two server threads, and without protection both threads can read the same queue and pick the same patient. We protect it with SQLite's database write lock [16]. BEGIN IMMEDIATE acquires the lock before the queue is read and COMMIT releases it, so the sequence behaves like code placed between wait(mutex) and signal(mutex). A second request simply blocks inside BEGIN IMMEDIATE until the first one commits (Fig. 8). The update also re-checks the status as a second line of defence:

```
UPDATE visits
SET status = 'in_consult',
    doctor_id = ?, consult_started_at = ?
WHERE visit_id = ? AND status = 'waiting';
```

If this statement changes zero rows, the transaction is rolled back. We chose the database lock over a Python threading.Lock because a Python lock only protects threads inside one process. A hospital deployment would run several worker processes behind a web server, and the database lock still works in that setting.

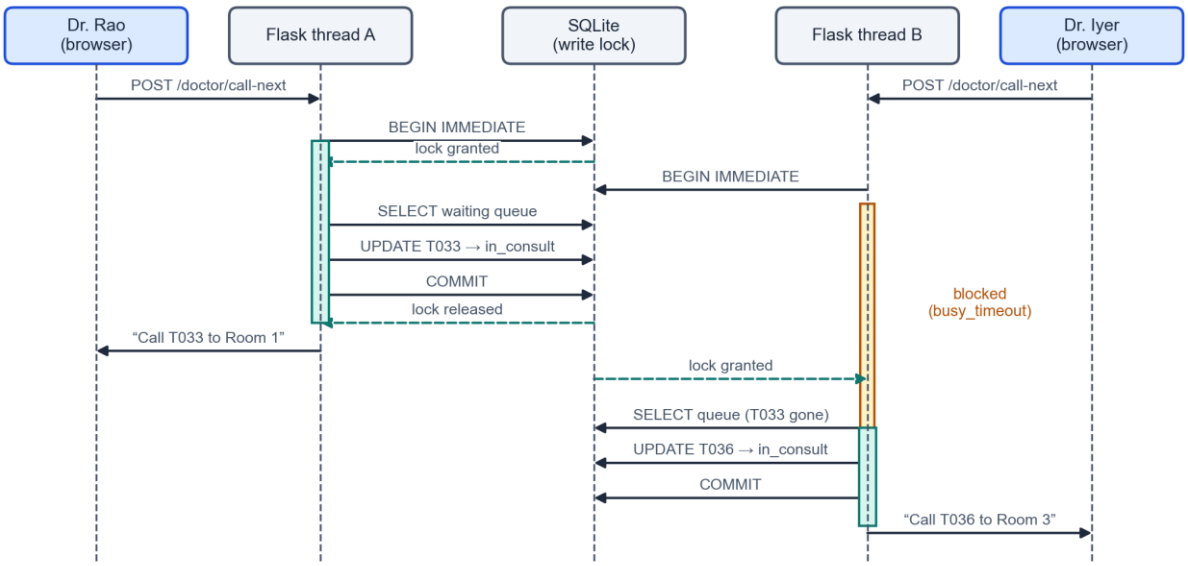


Fig. 8. Two doctors press "Call next" at the same moment. Thread B blocks until thread A commits, so each doctor gets a different patient.

C. Observation Beds as a Counting Semaphore

Admitted patients need one of eight observation beds, and the beds behave exactly like a counting semaphore whose value is the number of free beds (Fig. 9). When a doctor admits a patient, the system performs wait(): a free bed is taken if there is one, otherwise the patient joins the boarding queue, which is the semaphore's blocked queue. When housekeeping marks a bed as clean, the system performs signal(): the bed goes straight to the most urgent boarding patient, or back to the free pool if nobody is waiting. Handing the bed directly to the waiting patient, instead of incrementing the count and letting the next request grab it, stops a newly admitted patient from overtaking someone who has been boarding for an hour. A real ward needs time to clean a bed, so releasing a bed and signalling are two separate steps. Both run inside BEGIN IMMEDIATE … COMMIT, so they are atomic.

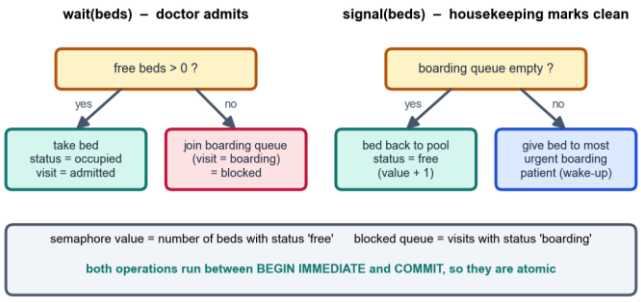


Fig. 9. wait() and signal() on the observation beds.

VI. AI Modules

A. Dataset

We did not have real triage records. MIMIC-IV-ED [9] needs credentialed access that we could not obtain within the semester, so we generated 3,000 synthetic patients. Each case first picks a visit type (minor injury, fever, abdominal pain, trauma, breathing difficulty, chest pain or stroke) with fixed frequencies, and then samples age, heart rate, systolic blood pressure, respiratory rate, SpO2, temperature, AVPU level and pain around values typical for that visit type. The label comes from rules based on NEWS2 [6] and the main ESI decision points [4]; for example, altered consciousness or SpO2 below 86% gives ESI 1, and chest pain above the age of 40 gives at least ESI 2. To imitate disagreement between nurses, about 6% of the labels were shifted by one level. The class counts are 292, 838, 736, 667 and 467 for ESI 1 to 5. We know that a model trained on rule-generated data mostly learns those rules back. The goal here was to build and test the complete pipeline, and the generator can be replaced by a real dataset without touching the rest of the code.

B. Triage Suggestion

Each case becomes a vector of 15 values: eight numeric inputs and a one-hot encoding of the seven complaints. A random forest [13] with 200 trees, a maximum depth of 12 and balanced class weights outputs the probability $P(k \mid x)$ of each level k. We do not show the plain argmax to the nurse. Under-triage, suggesting a less urgent level than the true one, is far more dangerous than over-triage, so a safety margin is applied to the most likely level $\hat{y}$:

$$\hat{y} \leftarrow \hat{y} - 1 \quad \text{if} \quad \hat{y} > 1 \text{ and } P(\hat{y} - 1 \mid x) \geq 0.30 \qquad (2)$$

On top of the model, red-flag rules force ESI 1 whatever the model says: SpO2 below 85%, systolic pressure below 80 mmHg, a patient who responds only to pain or not at all, heart rate above 150 or below 40, and respiratory rate of 35 or more or 8 or less. The nurse may change the suggestion but has to type a reason, and both levels are stored.

To show why a level was suggested, we use a simple what-if explanation in the spirit of LIME [18]. With the urgency score

$$U(x) = \Sigma\, P(k \mid x) \cdot (6 - k), \quad k = 1 \ldots 5 \qquad (3)$$

every abnormal input j is reset to a healthy reference value, giving $x^{(j)}$, and its effect is $\Delta_j = U(x) - U(x^{(j)})$. The largest effects, at most four, are shown as reasons such as "Complaint: chest pain, raises urgency". Our first version called the model once for each input and took about 52 ms. Sending all the variants to the model in a single batch brought the complete suggestion down to about 16 ms.

C. Consultation Time

A second random forest with 150 trees predicts the consultation time in minutes from the same features plus the final triage level, so it acts as the burst time of the process. The dispatcher does not use it for ordering, because shortest-job-first ordering made critical patients wait longer in our simulation (Section VIII-C). It feeds the waiting-time estimate and the progress bar on the doctor's console instead.

D. Waiting-Time Estimate

For each waiting patient, the server first replays the dispatcher forward in time. Every doctor on duty becomes free either now or when the predicted consultation of the current patient ends, and the queue is served in dispatch order. The replay cannot know about patients who have not arrived yet, so it is far too optimistic for low-priority patients, who keep being overtaken. We therefore trained a gradient boosting regressor [14] with 300 trees of depth 3 and an absolute-error loss. Its inputs are the replay estimate, the ESI level, the number of patients ahead on higher and equal levels, the number of busy and on-duty doctors, the arrivals in the last hour and the hour of day, and its target is the wait that actually happened. It was trained on 6,342 patients from 80 simulated shifts and tested on 1,950 patients from 25 different shifts. The waiting-room display shows the result as a range rather than a single number, because a single number tends to be read as a promise. Fig. 10 summarises the AI pipeline.

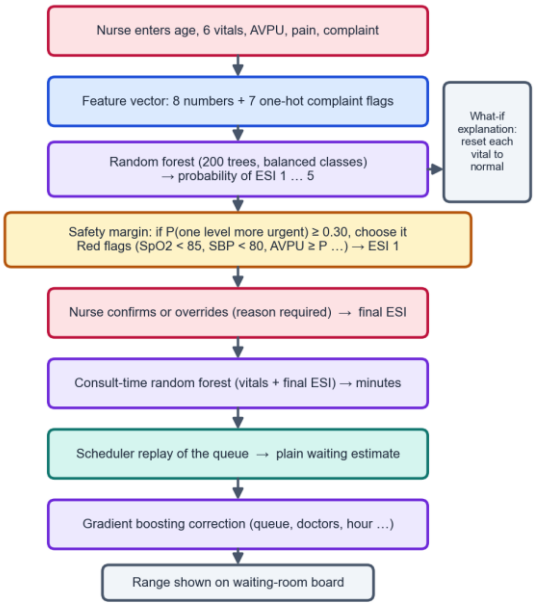


Fig. 10. AI pipeline from vital signs to the waiting-room display.

VII. Implementation

A. Technology and Structure

The project has about 2,900 lines of code. schema.sql creates the tables, indexes and a queue view. scheduler.py holds the pure scheduling logic (effective level, ordering, preemption target and the replay) and is shared by the web application and the offline simulator, so the policy we evaluated is exactly the policy that runs. dispatch.py and beds.py contain the critical sections, app.py has about 20 Flask routes, the ai package holds data generation, training and the runtime models, and simulate.py and race_test.py produce the evaluation results. SQL is written by hand so that every query is visible. A typical query, used for the daily statistics, is:

```
SELECT COUNT(*) AS seen,
  AVG((julianday(first_seen_at)
     - julianday(arrived_at)) * 1440)
     AS avg_wait
FROM visits
WHERE first_seen_at IS NOT NULL
  AND date(arrived_at) =
      date('now', 'localtime');
```

The complete source code is public on GitHub at https://github.com/anandaahan7-prog/mediqueue.

B. Security and Privacy

Passwords are stored as salted hashes generated by Werkzeug, every query uses parameter binding, and each route checks the user's role. The waiting-room display is the only page without a login, and it shows only token numbers, rooms and estimated waits; one of our automated tests checks that no patient name ever appears in its response. Every scheduling and bed decision is written to the events table, so any disputed decision can be traced to a person and a time.

C. Demonstration Data

The seed script creates two nurses, four doctors (three on duty), one administrator and eight observation beds. It fills the day with 26 completed visits, two patients in consultation and ten waiting patients, including an ESI 5 patient who has waited 78 minutes and has therefore aged to level 3, and one visit where the nurse overrode the AI. All timestamps are relative to the moment the script runs, so the queue always looks like a busy afternoon during a demonstration.

VIII. Results and Discussion

A. Functional Testing

We wrote 14 automated tests with Python's unittest module. Each test starts from a freshly seeded database, and all of them pass in about 28 seconds. Table IV lists the main cases.

TABLE IV.
SELECTED TEST CASES

Test case	Expected result	Status
Login with wrong password	Error, no session	Pass
Nurse opens admin page	Access denied	Pass
Heart-attack vitals	ESI 1 or 2 suggested	Pass
SpO2 82%, responds to pain	Red flag, ESI 1	Pass
Override without reason	Rejected	Pass
Doctor calls next	Gets head of queue	Pass
Call next while busy	Refused	Pass
ESI 5 waiting 78 min	Ranked above newer ESI 3	Pass
4 doctors call at once	4 different patients	Pass
ESI 1 arrives, all busy	ESI 3–5 consult paused	Pass
Worse vitals on re-triage	Level raised	Pass
Admit with no free bed	Boarding, then bed on clean	Pass
Waiting-room API	No patient names	Pass

B. Triage Models

Table V summarises the triage models under five-fold cross validation. The plain random forest is right 86.2% of the time and within one level 99.1% of the time, but it

under-triages 8.8% of the patients. With the safety margin and the red-flag rules, under-triage falls to 5.2%. The price is more over-triage (12.0% instead of 5.0%) and a lower exact accuracy of 82.8%. We think this is the right trade for an emergency department: an over-triaged patient is seen a little earlier than necessary, whereas an under-triaged one may deteriorate in the waiting room. The NEWS2-only baseline does much worse, with 55.3% accuracy and 41.5% under-triage, mainly because it ignores the complaint and the pain score; a stroke patient with normal vital signs scores almost zero on NEWS2.

TABLE V.
EVALUATION OF THE AI MODELS

Model / metric	**Result**
Plain forest: accuracy / macro F1	86.2% / 0.868
Plain forest: under / over-triage	8.8% / 5.0%
With safety rules: accuracy	**82.8%**
With safety rules: under / over-triage	**5.2% / 12.0%**
NEWS2 only: accuracy / under-triage	55.3% / 41.5%
Within ±1 level (plain forest)	99.1%
Consult time: MAE (mean 31.6 min)	4.0 min
Waiting time: MAE, replay only	13.3 min
Waiting time: MAE, with ML correction	8.9 min
Triage suggestion latency	≈16 ms

Fig. 11 shows the confusion matrix of the suggestions that the nurse actually sees. Most of the remaining errors are between neighbouring levels, and the matrix is visibly shifted towards the more urgent side, which is the intended effect of the safety margin. Fig. 12 shows that the complaint (all seven flags together), the pain score and SpO2 carry the most weight. The consultation-time model has a mean absolute error of 4.0 minutes on an average consultation of 31.6 minutes, and 71% of its predictions fall within 5 minutes.

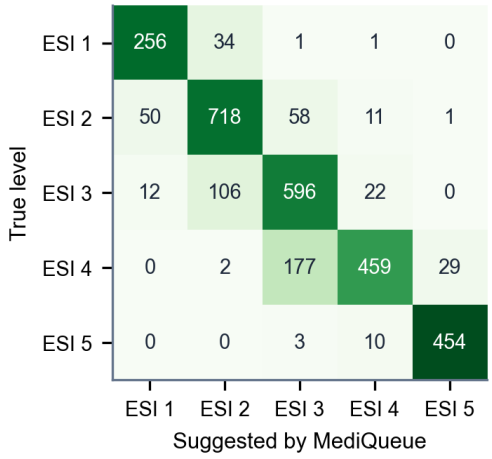


Fig. 11. Confusion matrix of the final suggestion (safety margin and red flags), 5-fold cross validation.

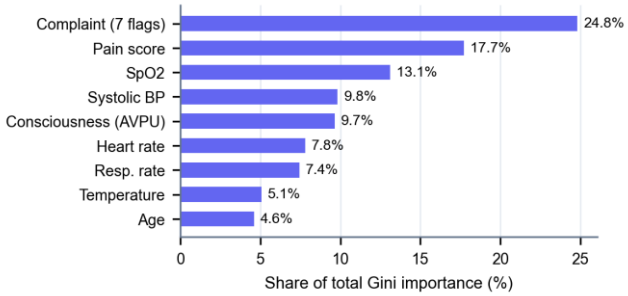


Fig. 12. Gini importance of the inputs of the triage forest.

C. Scheduling Policies

We compared four dispatch policies on 40 simulated shifts of twelve hours (3,174 patients) with four doctors and an arrival pattern with a late-morning and an evening peak. Every policy saw the same patients and only the AI's suggested level and predicted time, but the results are grouped by each patient's true level. Table VI and Fig. 13 show the main trade-off.

TABLE VI.
SCHEDULING POLICIES OVER 40 SIMULATED SHIFTS (MINUTES)

Policy	**Mean**	**ESI 1–2**	**95th pct ESI 4–5**	**ESI 1 on time**
FCFS (token)	28.1	27.0	98	31%
SJF (predicted)	19.6	36.2	27	33%
Priority only	42.0	10.3	355	37%
MediQueue	**36.3**	**9.4**	**206**	**80%**

FCFS makes ESI 1–2 patients wait 27.0 minutes on average, and SJF on predicted time is even worse for them (36.2 minutes) because serious cases take longer to treat. Plain priority scheduling brings critical waits down to 10.3 minutes but starves the bottom of the queue: the 95th percentile wait of ESI 4–5 patients reaches 355 minutes. MediQueue gives the lowest critical wait (9.4 minutes) and by far the best on-time rate for ESI 1 patients, 80% against 37% for plain priority, thanks to preemption. It also cuts the 95th percentile wait at the bottom of the queue to 206 minutes, with about four preemptions per shift.

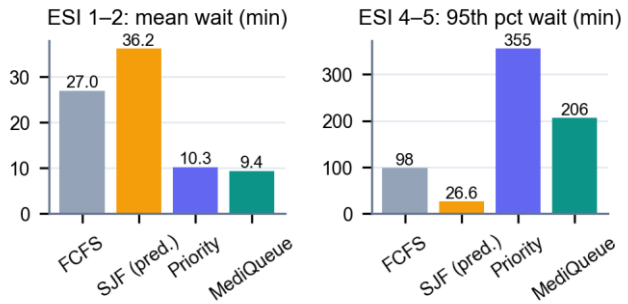


Fig. 13. Mean wait of critical patients and 95th percentile wait of low-priority patients for each policy.

MediQueue does not win everywhere. Its mean wait over all patients (36.3 minutes) is higher than FCFS (28.1) and SJF (19.6), and ESI 3 patients are seen on time slightly less often than under plain priority (63% against 68%), because aged ESI 4–5 patients now compete with them at level 3 (Fig. 14). The low mean wait of SJF is the textbook result of serving short jobs first, but it is achieved by

making the sickest patients wait, which is not acceptable in an emergency department.

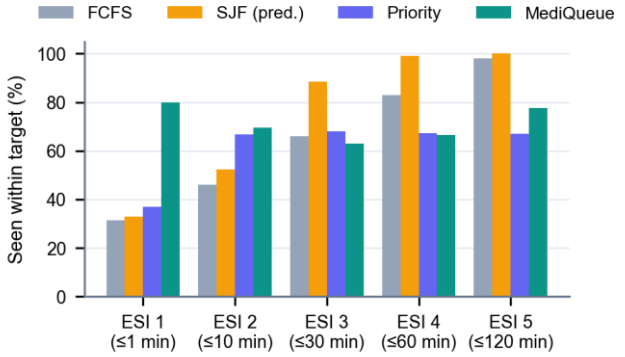


Fig. 14. Share of patients seen within the ESI target time, by true level.

Fig. 15 shows how we chose the aging interval. Between 5 and 30 minutes the tail wait of low-priority patients barely changes (158 to 160 minutes) while the share of ESI 3 patients seen on time rises from 62% to 65%. Beyond 30 minutes the tail starts to grow, and switching aging off raises it to 266 minutes. The administrator can change the interval if a hospital prefers a different balance.

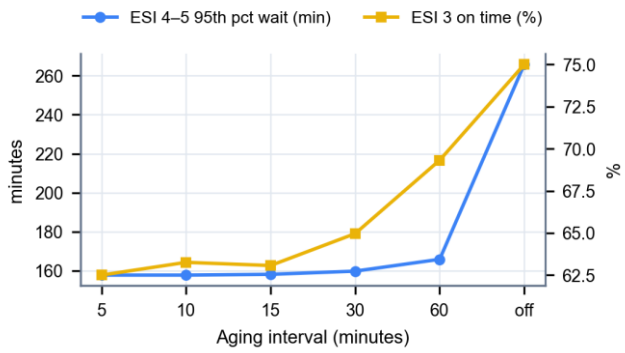


Fig. 15. Effect of the aging interval on low-priority tail wait and ESI 3 timeliness (25 shifts).

D. Waiting-Time Estimate

Fig. 16 compares estimated and actual waits for 400 test patients. The plain replay has a mean absolute error of 13.3 minutes and the ML correction 8.9 minutes, a reduction of about a third. For ESI 1–2 patients both estimates are already good, with errors below 4 minutes, and nearly all of the improvement comes from ESI 3–5 patients, whose error falls from 17–22 minutes to 12–15 minutes. The replay estimate itself remains the most important input of the correction model (84% of its importance), followed by the ESI level (9%) and the hour of day (4%). The median error is under 2 minutes for both methods, which means that most estimates are close and a minority are far off; showing a range on the display is our way of being honest about that.

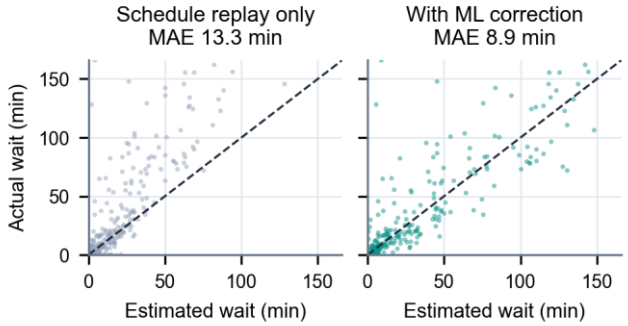


Fig. 16. Estimated against actual wait. Points on the dashed line are perfect estimates.

E. Concurrency Test

The stress test in race_test.py runs on a scratch database. In the first scenario six threads press “Call next” at the same instant with six patients waiting; in the second, eight threads admit patients while three beds are free. Each scenario was repeated 30 times, with a random pause of 0.5 to 4 ms between reading and writing to imitate request handling. Without the lock, 45 patients were given to two doctors and beds were handed to more than one patient 50 times (Fig. 17). With the critical section there were no conflicts at all. The price is latency: the average time per request rose from 22.0 to 34.5 ms for dispatch and from 30.3 to 48.1 ms for admissions, because requests now wait for each other. That is far below anything a doctor would notice.

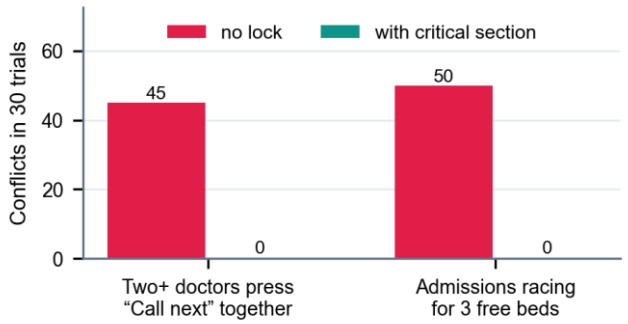


Fig. 17. Conflicts in 30 trials with and without the critical section.

F. User Interface

Fig. 18 shows the triage desk. As the nurse types, the AI panel updates with the suggested level, the expected consultation time, the NEWS2 score and the reasons, and abnormal vitals are outlined in red. Fig. 19 shows the queue as the nurse sees it, in the exact order the dispatcher will use; aged patients, nurse overrides and patients who need their vitals re-checked are marked. Fig. 20 shows the preemption banner on the console of the doctor whose patient is the least urgent, Fig. 21 the bed board with the semaphore value and the boarding queue, and Fig. 22 the waiting-room display, which shows tokens and ranges but no names.

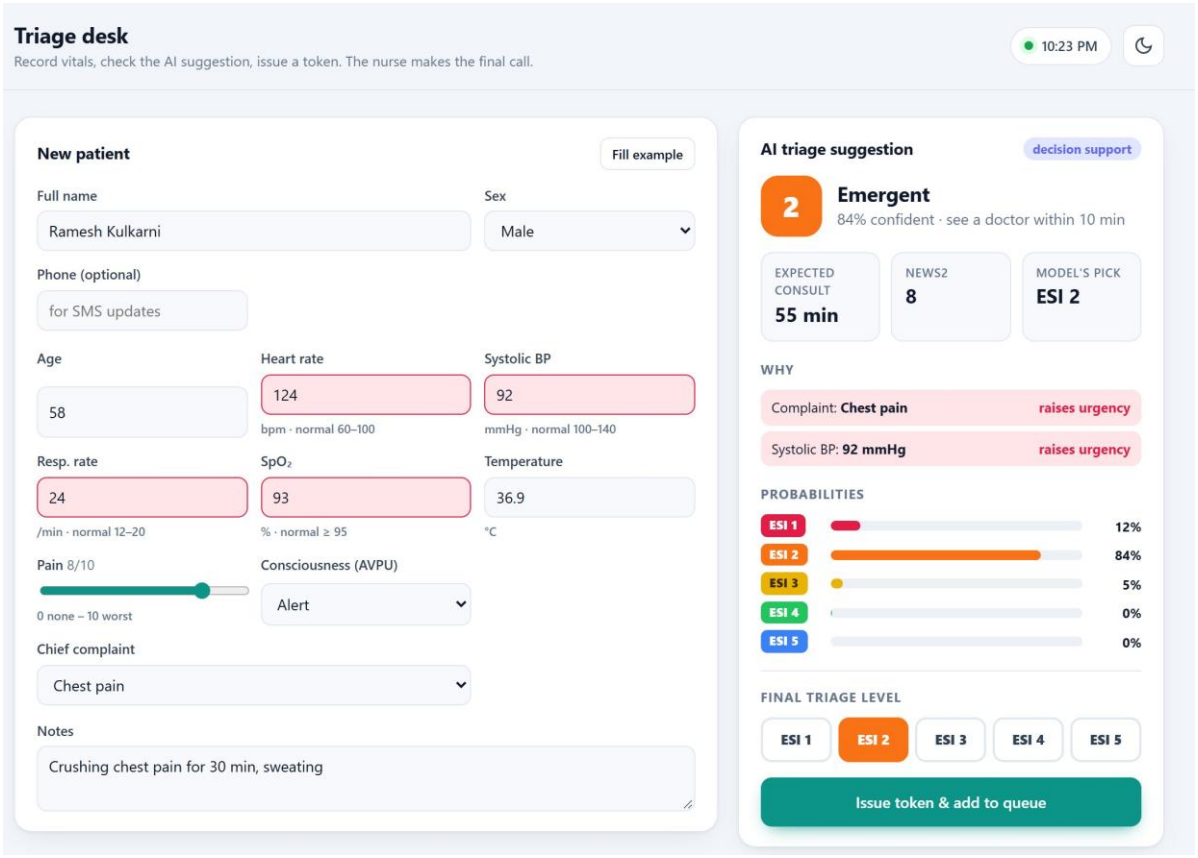


Fig. 18. Triage desk with the live AI suggestion and the final level chosen by the nurse.

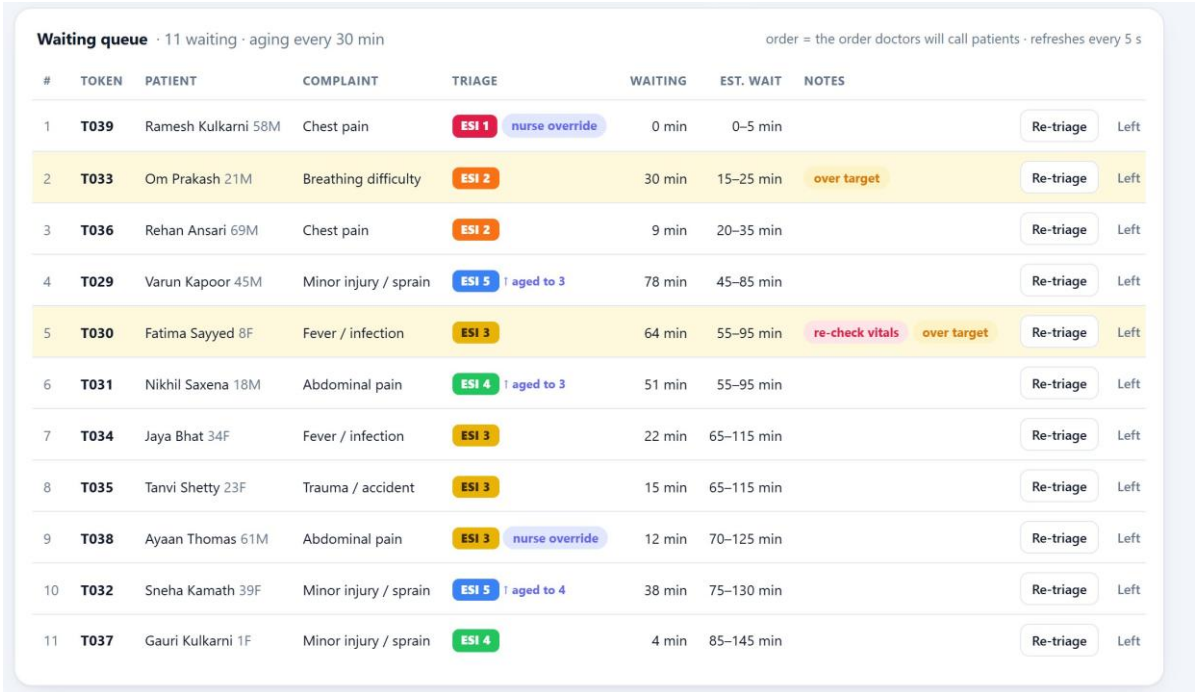


Fig. 19. Waiting queue in dispatch order, with aging, overrides and re-check reminders.

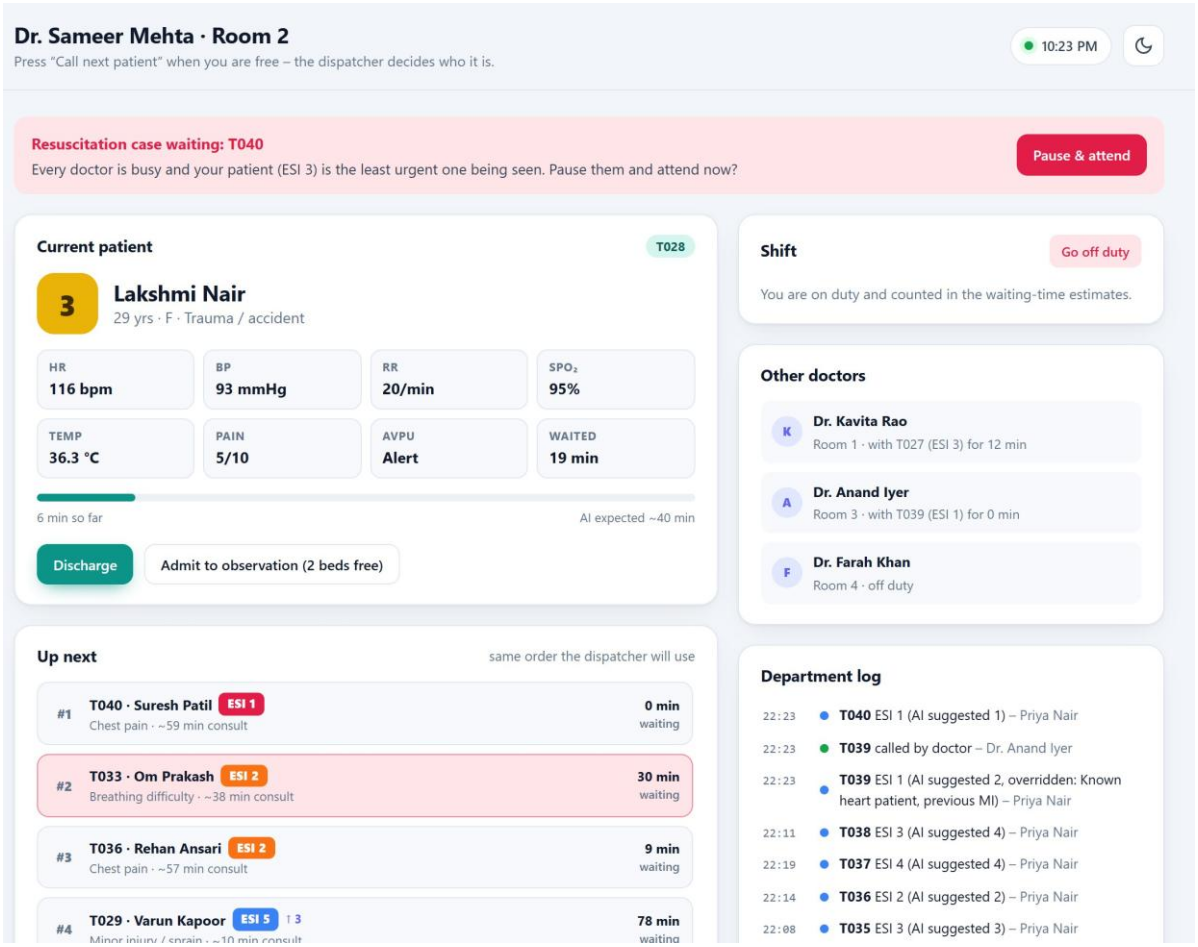


Fig. 20. Doctor console: an ESI 1 patient is waiting and this doctor is asked to pause an ESI 3 consultation.

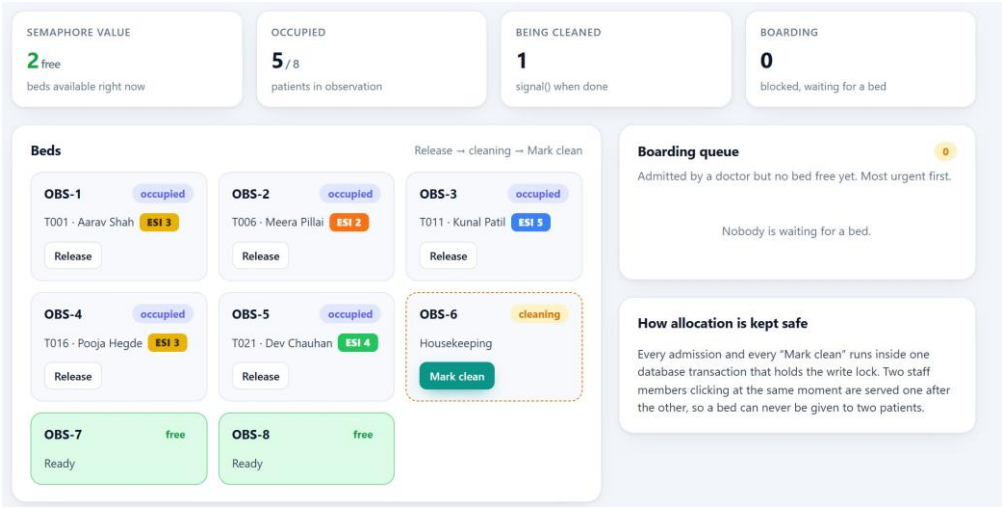


Fig. 21. Bed board: semaphore value, beds being cleaned and the boarding queue.

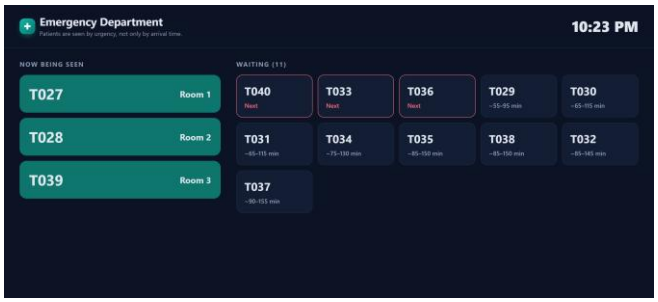


Fig. 22. Waiting-room display, refreshed every five seconds.

G. Limitations

The models were trained and evaluated on synthetic data, so the accuracy figures describe how well the pipeline learns our rules, not how it would perform on real patients. The waiting-time model was also trained on simulated shifts. Doctors are treated as identical processors, while a real department has specialities. The system has not been tried in a hospital, and it is meant as decision support, not as a medical device.

IX. Conclusion and Future Scope

MediQueue began as an attempt to show as many operating system concepts as possible and ended up using fewer of them, but each one now does a job that a real emergency department needs. Priority scheduling with capped aging gets critical patients to a doctor faster than a token queue while keeping low-priority patients from waiting indefinitely, and limiting preemption to resuscitation cases keeps interruptions to about four per shift. Treating dispatch and bed allocation as critical sections removed every double assignment in our stress test at a cost of roughly fifteen milliseconds per request. The AI models are deliberately modest and explainable: they suggest, the nurse decides, and every disagreement is recorded.

Future work includes training the models on MIMIC-IV-ED once access is approved, retraining the waiting-time model on the hospital's own logged visits, modelling doctor specialities as heterogeneous processors (for example paediatric and trauma rooms), sending SMS updates to the phone number collected at triage, and integrating with hospital information systems through HL7 FHIR. Most importantly, the system needs a pilot with real nurses, whose overrides would tell us where the model is wrong.

Acknowledgment

The authors thank Dr. Ezhil Arasi V, Department of Computer Science and Engineering, Vellore Institute of Technology, for guidance and support throughout this project.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018.

[2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Boston: Pearson, 2015.

[3] E. W. Dijkstra, “Cooperating sequential processes,” Technological University, Eindhoven, Tech. Rep. EWD-123, 1965.

[4] N. Gilboy, P. Tanabe, D. Travers, and A. M. Rosenau, Emergency Severity Index (ESI): A Triage Tool for Emergency Department Care, Version 4, Implementation Handbook. Rockville, MD: AHRQ, Publication No. 12-0014, 2012.

[5] L. Kleinrock, Queueing Systems, Volume II: Computer Applications. New York: Wiley, 1976.

[6] Royal College of Physicians, National Early Warning Score (NEWS) 2: Standardising the Assessment of Acute-Illness Severity in the NHS. London: RCP, 2017.

[7] S. Levin et al., “Machine-learning-based electronic triage more accurately differentiates patients with respect to clinical outcomes compared with the Emergency Severity Index,” Ann. Emerg. Med., vol. 71, no. 5, pp. 565–574, 2018.

[8] W. S. Hong, A. D. Haimovich, and R. A. Taylor, “Predicting hospital admission at emergency department triage using machine learning,” PLoS ONE, vol. 13, no. 7, e0201016, 2018.

[9] A. Johnson, L. Bulgarelli, T. Pollard, L. A. Celi, R. Mark, and S. Horng, “MIMIC-IV-ED (version 2.2),” PhysioNet, 2023.

[10] E. Ang, S. Kwasnick, M. Bayati, E. L. Plambeck, and M. Aratow, “Accurate emergency department wait time prediction,” Manuf. Serv. Oper. Manag., vol. 18, no. 1, pp. 141–156, 2016.

[11] S. Saghafian, G. Austin, and S. J. Traub, “Operations research/management contributions to emergency department patient flow optimization: Review and research prospects,” IIE Trans. Healthc. Syst. Eng., vol. 5, no. 2, pp. 101–123, 2015.

[12] S. Saghafian, W. J. Hopp, M. P. Van Oyen, J. S. Desmond, and S. L. Kronick, “Complexity-augmented triage: A tool for improving patient safety and operational efficiency,” Manuf. Serv. Oper. Manag., vol. 16, no. 3, pp. 329–345, 2014.

[13] L. Breiman, “Random forests,” Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[14] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Ann. Statist., vol. 29, no. 5, pp. 1189–1232, 2001.

[15] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[16] SQLite Consortium, “File locking and concurrency in SQLite version 3.” [Online]. Available: https://www.sqlite.org/lockingv3.html

[17] M. Grinberg, Flask Web Development, 2nd ed. Sebastopol, CA: O’Reilly, 2018.

[18] M. T. Ribeiro, S. Singh, and C. Guestrin, “‘Why should I trust you?’: Explaining the predictions of any classifier,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 1135–1144.